Microsoft Purview Center of Excellence

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Microsoft Purview Center of Excellence 5

What you get . . . 5

What this delivers . . . 5

The deployment model . . . 6

Scope boundary . . . 6

Outcome . . . 6

1. Solution Overview — Microsoft Purview Center of Excellence 6

What this solution is . . . 6

Why this matters . . . 7

How it works . . . 7

The CoE governance board (least-privilege, separated) . . . 8

The CoE standards (the house rules) . . . 9

The intake workflow (request to production) . . . 9

The lifecycle cadence . . . 9

What you can control . . . 9

Limitations (be honest with buyers) . . . 10

Naming convention (used in every doc and script) . . . 10

The standard deployment model . . . 10

2. Requirements Guide — Microsoft Purview Center of Excellence 11

2.1 Compliance licensing inventory (what the CoE can govern) . . . 11

2.2 Least-privilege governance access (Compliance Administrator, not blanket Global Admin) . . . 11

2.3 Security & Compliance PowerShell + Exchange Online + Graph toolchain . . . 11

2.4 Graph read scopes (read-first, no write scopes for governance reads) . . . 12

2.5 A named CoE governance board (least-privilege, separated) and role-group matrix . . . 12

2.6 The CoE standards, agreed and captured . . . 12

2.7 An intake workflow and change control . . . 13

2.8 A complete, readable inventory of governed Purview artifacts . . . 13

2.9 Stakeholders and the determination boundary . . . 13

2.10 A pilot artifact in simulation/test . . . 13

Checklist . . . 13

Global Requirements — Microsoft Purview Solutions 14

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 14

2. Purview roles (least-privilege) and portal access . . . 15

3. Security & Compliance PowerShell + Graph toolchain . . . 15

4. Data sources & scope in scope . . . 15

5. Compliance, records, privacy & legal stakeholders . . . 15
6. A pilot scope and change control . . . 16
3. Implementation Guide — Microsoft Purview Center of Excellence 16
Phase 0 — Assess (read-only) . . . 16
Phase 1 — Design (author the operating model) . . . 17
Phase 2 — Configure (pilot-first, `-WhatIf` before `-Commit`) . . . 17
Phase 3 — Pilot the intake gate (simulation/test) . . . 18
Phase 4 — Validate (see the Testing Guide for full procedures) . . . 19
Phase 5 — Expand (run the CoE as a standing rhythm) . . . 19
Rollback summary . . . 20
5. Testing & Validation Guide — Microsoft Purview Center of Excellence 20
Test 1 — Scope and toolchain (Requirement 2.1–2.4) . . . 20
Test 2 — Estate inventory completeness (Requirement 2.8) . . . 20
Test 3 — Board least-privilege and separation of duties (Requirement 2.5) . . . 21
Test 4 — Naming-standard conformance (Requirement 2.6) . . . 21
Test 5 — Ownership and review-date coverage (Requirement 2.6) . . . 21
Test 6 — Simulation-first gate (Requirement 2.10, the highest-value test) . . . 21
Test 7 — Intake gate end-to-end on the pilot (Requirement 2.10) . . . 21
Test 8 — Register reconciles to the live tenant (drift) . . . 22
Test 9 — Scorecard produced and reconciles (Requirement 2.9) . . . 22
Regression / cadence checklist (run every review cycle) . . . 22
6. Troubleshooting — Microsoft Purview Center of Excellence 22
`OPSCOE-ScopeCheck.ps1` can't see a capability you intend to govern . . . 22
`OPSCOE-ApplyStandards.ps1 -RoleMatrixCsv` prints `[MISSING] role group` . . . 23
The preview would add someone to an investigative role . . . 23
The ownership stamp changed (or seems to have changed) a policy . . . 23
Stamp skips a capability with `[SKIP] … cmdlets unavailable` . . . 23
`OPSCOE-Validate.ps1` reports FAIL on naming . . . 23
Validate reports FAIL on `EnforceEvidence` . . . 24
Validate reports FAIL on `Owner` / `ReviewDate` . . . 24
Separation-of-duties FAIL on the board matrix . . . 24
Reconciliation reports `Unregistered` / orphaned live artifacts . . . 24
`OPSCOE-Register.ps1` scorecard shows enforced-without-evidence > 0 . . . 24
`Connect-IPPSSession` / `Connect-ExchangeOnline` / `Connect-MgGraph` fails . . . 25
When to escalate beyond this solution . . . 25
7. Best Practices — Microsoft Purview Center of Excellence 25

Govern the operating model, not the determinations . . . 25
Make simulation-first non-negotiable . . . 25
Least privilege and separation of duties, enforced on the CoE itself . . . 26
Standardize names so the estate reads itself . . . 26
Stamp ownership onto the object where you can . . . 26
Run a real cadence, or the register rots . . . 26
Keep the CoE lightweight enough to survive . . . 27
Assume the portal will change . . . 27
Protect the evidence and the credentials . . . 27
8. Deliverables Summary — Microsoft Purview Center of Excellence 27
Documentation . . . 27
PowerShell scripts . . . 28
What the buyer receives . . . 29
Outcome . . . 29
Appendix A — PowerShell Scripts 30

Executive Summary — Microsoft Purview Center of Excellence

Microsoft Purview is powerful and sprawling, and once the platform is turned on, the next failure mode is **governance drift**: labels, DLP policies, and retention rules created ad hoc by whoever held the role that day, none of them owned, none sharing a naming or lifecycle standard, high-impact policies flipped to *enforce* without ever running in **simulation/test**, and — six months on — no one able to say which policy does what, who approved it, or when it was last reviewed. That is a compliance gap and a data-risk exposure.

This solution institutionalizes Purview governance by standing up a **Purview Center of Excellence (CoE)** — the operating model that governs every Purview capability from one place: a named, least-privilege **governance board** with separation of duties; written **standards** (naming, ownership + review cadence, a mandatory simulation/test-first rule, change control, and separation-of-duties); a single **intake** workflow that takes every request Request → Triage → Design → Pilot in simulation → Validate → Board approval → Enforce → Register; and a **lifecycle** cadence with drift detection and a deliberate decommission path. Everything is done with the Microsoft Purview portal, the Microsoft 365 admin center, Security & Compliance PowerShell, Exchange Online PowerShell, and Microsoft Graph — no third-party tooling.

This is a complete, deploy-it-today package for institutionalizing Purview governance, written for a competent M365/compliance administrator and the governance board they support.

What you get

• A plain-language Solution Overview of the CoE operating model — the governance board and role model,

the standards, the intake workflow, and the lifecycle cadence.

• A Requirements Guide with verify / implement-if-missing for every prerequisite, including which

capabilities your licensing actually puts in the CoE's remit.

• An Implementation Guide of precise, ordered portal + PowerShell + Graph steps, with the board role

model and standard tags applied pilot-first (`-WhatIf` before `-Commit`) and the intake gate proven in simulation before the estate is governed.

• Five PowerShell scripts: a scope/health helper, an estate-inventory discovery, a standards/role

configuration helper (`-WhatIf` by default; writes only role membership and a non-disruptive metadata tag), a governance-gate validator, and a governance scorecard/register report.

• Testing and Troubleshooting guides for a confident, safe rollout.

What this delivers

A named, least-privilege, separated governance board; captured standards; a complete register of every governed artifact with an owner and a review date; a proven simulation-first intake gate (**enforced-without-evidence = 0**); and a standing lifecycle cadence with drift detection — validated on a pilot artifact in simulation before the full estate is brought under governance.

The deployment model

Assess → Design → Configure → Pilot → Validate → Expand.

Scope boundary

The CoE governs the **operating model** — board, standards, intake, lifecycle, and evidence. It does not author policy contents and does not make regulatory, records, or legal determinations: retention schedules, records file plans, legal holds, and insider-risk/communication-compliance case actions are deferred to the organization's compliance, records, privacy, and legal functions. High-impact policies are always piloted in simulation/test/disabled mode before enforcement, and the CoE never flips a policy to enforce — that remains a board-gated, intake-driven action taken after simulation passes.

Outcome

Purview governance institutionalized — a governance board, standards, intake, and lifecycle in place, piloted, and validated — so every governed artifact is owned and defensible and the audit answer is one scorecard export, delivered and defensible in Microsoft Purview.

1. Solution Overview — Microsoft Purview Center of Excellence

What this solution is

Microsoft Purview is broad — sensitivity and retention labels, data loss prevention (DLP), data lifecycle and records management, insider risk, communication compliance, audit, eDiscovery, data security posture management (DSPM), and data governance. Once a tenant has the *foundation* in place, the next failure mode is **governance drift**: labels, DLP policies, and retention rules get created ad hoc by whoever had the role that day, nobody owns them, none of them share a naming or lifecycle standard, high-impact policies get flipped to *enforce* without ever running in **simulation/test**, and six months later nobody can say which policy does what, who approved it, or when it was last reviewed. That is a compliance gap and a data-risk exposure.

This solution institutionalizes Purview governance by standing up a **Purview Center of Excellence (CoE)** — the operating model that governs every Purview capability from one place:

- A **governance board** — a named body with a charter, a RACI, and least-privilege membership mapped

to built-in Purview role groups (compliance, information protection, records, insider risk, communication compliance, eDiscovery, audit) with **separation of duties** enforced.

- **Standards** — the house rules every Purview artifact must follow: a **naming convention**, a

mandatory **owner + review cadence**, a **simulation/test-first** rule for every high-impact policy, a **change-control** rule (every change via intake, versioned, reversible), and a **separation-of-duties** rule for HR/legal-sensitive capabilities.

- **Intake** — a single request-to-production workflow (Request → Triage → Design → Pilot in

simulation/test → Validate → Board approval → Enforce → Register) so nothing lands in the tenant without an owner, a standard-conformant name, recorded simulation evidence, and board sign-off.

• **Lifecycle** — a periodic review cadence, **drift detection** (current configuration compared to

the registered standards baseline), versioning, and a decommission path for orphaned or unowned artifacts — so the estate stays understood and defensible over time.

Everything is done with the **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), Exchange Online PowerShell (for audit config), and **Microsoft Graph** — no third-party tooling. The five PowerShell scripts inventory every governed artifact, apply the CoE role model and standards tags (`-WhatIf` first), validate governance gates, and produce the governance register/scorecard.

> ***This is the governance operating model, not the capability build-outs.*** *The CoE governs the *labels, DLP policies, retention/records rules, insider-risk and communication-compliance policies, eDiscovery cases, and DSPM* that other Purview solutions configure. It does not author those policies' contents or make the regulatory/records/legal determinations behind them — those are deferred to the organization's compliance, records, privacy, and legal functions. The CoE supplies the board, the standards, the intake gate, the lifecycle cadence, and the evidence.*

Why this matters

• **Ungoverned Purview drifts into risk.** Every unowned DLP or retention policy is a change nobody

can explain and nobody dares touch. A CoE gives every artifact an owner, a standard-conformant name, and a review date — so the estate stays auditable.

• **Simulation-first is the single most valuable house rule.** DLP, auto-labeling, retention, and

insider-risk policies can block mail, encrypt files, delete content, and flag employees. Making **simulation/test mode a mandatory intake gate** — with recorded evidence before any policy is enforced — prevents the most damaging Purview incidents.

• **Separation of duties is compliance-critical.** Insider risk, communication compliance, and

eDiscovery expose sensitive employee content. The CoE keeps those role groups separate, membership minimal and named, and the *determination* of who may hold them deferred to HR, privacy, and legal.

• **A register turns Purview into a defensible program.** When an auditor asks "what governs your

sensitive data, who owns it, and when was it last reviewed?", the governance register answers in one export instead of a scramble.

How it works

1. **Assess.** Read the tenant's compliance licensing (which capabilities the CoE can actually

govern), the existing role-group posture, and a full inventory of Purview artifacts — labels, label policies, DLP policies, retention policies/labels, auto-labeling, and (where licensed and readable) communication-compliance and audit-retention policies. Read-only, no changes.

1. **Design.** Author the **CoE charter and board RACI**, the **standards** (naming, ownership, review

cadence, simulation-first, change-control, separation of duties), and the **intake workflow**. Map board functions to least-privilege built-in role groups.

1. **Configure.** Create/populate the CoE governance role groups (least-privilege, from a matrix), register the standards baseline, and optionally stamp each artifact with an ownership/standard tag — role and tag changes applied with a `-WhatIf` **preview first**.
1. **Pilot.** Run the intake workflow end-to-end on a **pilot artifact** in **simulation/test mode** (for example, one DLP or auto-labeling policy taken Request → simulate → validate → board approval) to prove the gate works before governing the whole estate.
1. **Validate.** Confirm every governance gate: naming conformance, every artifact has an owner and a review date, no high-impact policy is enforced without recorded simulation evidence, separation of duties intact, and the register reconciles to the live tenant.
1. **Expand.** Bring the full artifact estate under the register and the review cadence; run the CoE as a standing operating rhythm (intake for new requests, periodic lifecycle reviews, drift detection).

The CoE governance board (least-privilege, separated)

Map each board function to the **built-in Purview/compliance role group** — add named members; create custom scoped role groups only where tighter scoping is required.

Board function	Built-in role group	Separation
Chair / compliance administration	**Compliance Administrator**	Governs; does not hold investigative roles
Information protection (labels/DLP standards owner)	**Information Protection Admin**	—
Classification data	**Compliance Data Administrator**	—
Records & retention standards owner	**Records Management**	Determinations deferred to records/legal
Insider risk representative	**Insider Risk Management**	**Separated** — HR/security; minimal, named
Communication compliance representative	**Communication Compliance**	**Separated** — HR/compliance; minimal, named
eDiscovery / legal representative	**eDiscovery Manager**	**Separated** — legal/investigations
Audit representative	**Audit Manager** / **Audit Reader**	Read-only where sufficient

> ***Separation of duties is a standard, not a nicety.*** *The chair governs the operating model but does not hold insider-risk, communication-compliance, or eDiscovery roles. Those seats are separate, minimal, named, and — for insider risk and communication compliance — privacy-by-design (pseudonymization where the capability supports it) is applied in that capability's own solution. The CoE governs *how* these capabilities are stood up and reviewed;* ***who may hold the roles, and every regulatory/records/legal determination, is deferred to HR, privacy, records, and legal.***

The CoE standards (the house rules)

Standard	Rule
Naming	Every governed artifact follows `<Purpose> – <Capability> – <State>` (e.g. `Financial Data – DLP – Simulation`). Names carry purpose and state at a glance.
Ownership	Every artifact has a **named owner** and a **review cadence** (default quarterly), recorded in the register (and, where the object supports it, stamped in its `Comment`).
Simulation-first	No DLP, auto-labeling, retention, or insider-risk policy is enforced without first running in **simulation/test/disabled** mode with recorded pass evidence.
Change control	Every change enters via **intake**, is **versioned**, and has a documented **rollback**. No out-of-band edits.
Separation of duties	Insider-risk, communication-compliance, and eDiscovery roles are separate from governance administration; membership minimal and named.
Evidence	Every enforced policy has an evidence record (simulation results, board approval, register entry) — the audit answer.

The intake workflow (request to production)

`Request → Triage (board) → Design (standard-conformant name + owner) → Pilot in simulation/test → Validate (pass/fail gates) → Board approval → Enforce → Register (owner + review date)`. Every stage has an artifact (`Purview CoE Intake – Request`, simulation evidence, `Purview CoE Register – Governed`); nothing reaches *enforce* without passing simulation and board sign-off.

The lifecycle cadence

A standing rhythm: **periodic review** (default quarterly) of every registered artifact against the standards; **drift detection** (compare current tenant configuration to the registered standards baseline and flag new/changed/orphaned artifacts); **versioning** of every change; and a **decommission** path for orphaned or unowned policies (identified → owner-confirmed → disabled → removed under change control).

What you can control

- The CoE board membership (least-privilege, named, separated) and the role-group mapping.
- The standards themselves (naming pattern, review cadence, which capabilities are in remit — bounded

by licensing).

- The intake workflow gates and who approves.
- The lifecycle cadence (review frequency, drift-detection schedule, decommission policy).
- The pilot artifact used to prove the intake gate before governing the full estate.

Limitations (be honest with buyers)

- **The CoE governs; it does not determine.** Retention schedules, records file plans, legal holds, insider-risk and communication-compliance case actions, and regulatory applicability are **deferred** to compliance, records, privacy, and legal. The CoE provides the method, the gate, and the evidence.
- **Purview is heavily licensing-gated.** Advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, and DSPM require **Microsoft 365 E5** or the **E5 Compliance** add-on; some capabilities are in preview. The CoE governs what your plan unlocks — **verify availability against your own tenant and plan.**
- **The portal is evolving, and some capabilities have limited PowerShell.** Microsoft is actively changing the Purview portal and moving parts of role management to a unified Purview RBAC model. Insider Risk (and some communication-compliance) management is primarily **portal-managed** with limited/evolving PowerShell surface. Portal paths and cmdlets are cited as of authoring and flagged **"verify in your tenant"**; the scripts guard for cmdlet availability and never invent cmdlets.
- **The CoE does not author policy contents.** It governs naming, ownership, lifecycle, and the simulation-first gate — the *contents* of each label, DLP rule, and retention schedule are the subject of the capability-specific Purview solutions and the stakeholders' determinations.

Naming convention (used in every doc and script)

Governance artifacts this solution creates or captures follow `<Purpose> – CoE – <Stage>`; governed Purview artifacts follow the CoE **naming standard** `<Purpose> – <Capability> – <State>`.

Artifact	Name
CoE charter + board RACI matrix	`Purview CoE Charter – Board`
Standards baseline (the house rules, captured)	`Purview CoE Standards – Baseline`
Governed-artifact inventory / register	`Purview CoE Register – Governed`
Intake request record	`Purview CoE Intake – Request`
Lifecycle / drift review log	`Purview CoE Lifecycle – Review`
Governance scorecard	`Purview CoE Scorecard – Report`
CoE pilot artifact (simulation)	`Purview CoE Pilot – Stage1`

The standard deployment model

Assess the estate → design the board, standards, and intake → configure the role model and standards (pilot-first) → pilot the intake gate in simulation → validate every governance gate → expand to the full estate and run the lifecycle cadence. Pattern: ***Assess*** *→* ***Design*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand.***

2. Requirements Guide — Microsoft Purview Center of Excellence

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-purview.md` (the six shared requirements: compliance licensing, least-privilege Purview roles, the S&C PowerShell + Graph toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope with change control) — those are referenced here, not repeated.

> ***Foundation assumed.*** *This solution governs a tenant where the Purview foundation is already in place (portal organized, core services on, unified audit ingesting, workloads connected). If that foundation is missing, stand it up first — the CoE is the *operating model* layered over a working Purview tenant, not the initial enablement.*

2.1 Compliance licensing inventory (what the CoE can govern)

Description: A confirmed inventory of the tenant's **compliance licensing** — E3 for baseline capabilities, **Microsoft 365 E5** or the **E5 Compliance** add-on for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium (long-term retention), and DSPM. **Why:** The CoE's remit is bounded by what the plan unlocks; you cannot govern a capability the tenant cannot run (Global Requirement 1). **Verify:** Microsoft 365 admin center → `Billing` → `Licenses` shows the subscriptions/add-ons; run `OPSCOE-ScopeCheck.ps1` — it reports the compliance SKUs present (via Graph) and which governed capabilities are visible. Confirm capabilities appear in the Purview portal. **Implement:** Record each licensed capability as **in remit** and each unlicensed one as **out-of-scope for now** — do not attempt to govern a capability the tenant cannot use.

2.2 Least-privilege governance access (Compliance Administrator, not blanket Global Admin)

Description: An account holding **Compliance Administrator** to read the estate and manage role groups, plus a named **Global Administrator** reserved only for the one-time actions that genuinely require it (e.g. consenting Graph scopes). Everyday CoE work uses the least-privilege compliance role, per Global Requirement 2. **Why:** The CoE exists to enforce least privilege and separation of duties; running it from an over-granted Global Admin contradicts the standard it publishes. **Verify:** Purview portal → `Settings` → `Roles & scopes` → `Role groups` (verify path in your tenant) — confirm your account is in **Compliance Administrator**. `Connect-IPPSSession` succeeds and `Get-RoleGroup` returns. **Implement:** Add the account to **Compliance Administrator**; reserve a named Global Administrator only for consent/one-time setup.

2.3 Security & Compliance PowerShell + Exchange Online + Graph toolchain

Description: A workstation with the **ExchangeOnlineManagement** module (provides `Connect-IPPSSession` for Security & Compliance PowerShell **and** `Connect-ExchangeOnline` for the audit-config cmdlets and role-group membership) and the **Microsoft Graph PowerShell SDK**, with a modern-auth/MFA admin account (Global Requirement 3). **Why:** Every script runs through S&C PowerShell (labels/DLP/retention/role groups), Exchange Online PowerShell (audit config, `Add-RoleGroupMember`), and/or Graph (licensing, directory). **Verify:** `Get-Module -ListAvailable ExchangeOnlineManagement, Microsoft.Graph.Authentication`; `Connect-IPPSSession` and `Connect-MgGraph` both succeed. `OPSCOE-ScopeCheck.ps1` reports connectivity. **Implement:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser` and `Install-Module Microsoft.Graph -Scope CurrentUser`; confirm modern-auth sign-in.

2.4 Graph read scopes (read-first, no write scopes for governance reads)

Description: Delegated Graph scopes for the read-only inventory and scorecard: `Organization.Read.All`, `Directory.Read.All`, `User.Read.All`, `RoleManagement.Read.Directory`. No write scopes are required by the governance scripts. **Why:** The scope, discovery, and register scripts read licensing, directory, and role data through Graph; keeping to read scopes matches the safe-by-default posture. **Verify:** Run `OPSCOE-ScopeCheck.ps1 -UseGraph` (read-only) — it connects with the read scopes and reports if consent is missing. **Implement:** Consent at first `Connect-MgGraph`; if admin consent is required, a Global Administrator approves the scopes once in Entra (`Identity → Applications → Enterprise applications`).

2.5 A named CoE governance board (least-privilege, separated) and role-group matrix

Description: A named **governance board** — chair plus function representatives for information protection, records, insider risk, communication compliance, eDiscovery, and audit — mapped to **built-in Purview role groups** with **separation of duties** (insider risk / communication compliance / eDiscovery separate from governance administration), captured as a **role-group matrix** (CSV). **Why:** The board is the decision body of the CoE; its least-privilege, separated membership is the first standard the CoE enforces on itself (Global Requirements 2 and 5). **Verify:** The matrix names each seat and its role group; `OPSCOE-ApplyStandards.ps1 -WhatIf` previews the membership changes without applying them; `Get-RoleGroupMember` shows current membership. **Implement:** Author `Purview CoE Charter – Board` and the matrix CSV; populate role groups with `OPSCOE-ApplyStandards.ps1 -Commit` (adds named members to built-in role groups; creates custom scoped groups only where needed).

2.6 The CoE standards, agreed and captured

Description: The **house rules** agreed and written down: the **naming convention** (`<Purpose> – <Capability> – <State>`), the **ownership + review-cadence** rule (default quarterly), the **simulation/test-first** rule, the **change-control/versioning** rule, and the **separation-of-duties** rule — captured as `Purview CoE Standards – Baseline`. **Why:** Standards are what `OPSCOE-Validate.ps1` checks against; without a captured baseline there is nothing to validate conformance or detect drift against. **Verify:** The standards baseline exists and names the pattern, cadence, and gates. `OPSCOE-Validate.ps1` loads it and evaluates conformance. **Implement:** Agree the standards with compliance/records/privacy/legal stakeholders (Global Requirement 5); capture them as the standards baseline before governing the estate.

2.7 An intake workflow and change control

Description: A defined **intake workflow** (Request → Triage → Design → Pilot in simulation → Validate → Board approval → Enforce → Register) and a **change-control** process with a change window and approval path (Global Requirement 6). **Why:** Intake is the gate that keeps ungoverned, unowned, or un-simulated policies out of the tenant; change control makes every change reversible and auditable. **Verify:** The workflow is documented with an owner per stage; a change window/approval path exists; an intake request template (`Purview CoE Intake – Request`) is ready. **Implement:** Document the workflow and template; agree change control before enforcing any governed artifact.

2.8 A complete, readable inventory of governed Purview artifacts

Description: Read access to the artifacts the CoE will govern — **sensitivity labels and label policies**, **DLP policies and rules**, **retention policies/labels and rules**, **auto-labeling policies**, and (where licensed and readable) **communication-compliance policies** and **audit retention policies** — so the register can be built from the live tenant. **Why:** The register and drift detection are only as complete as what the account can read; some capabilities (insider risk especially) are portal-managed with limited PowerShell and are governed by attestation rather than enumeration. **Verify:** `OPSCOE-Discovery.ps1` (read-only) enumerates each readable artifact type and flags which are unlicensed or portal-only in this tenant. **Implement:** Ensure the account holds the least-privilege read roles for each in-remit capability; for portal-only capabilities, record them in the register by attestation with a named owner.

2.9 Stakeholders and the determination boundary

Description: Engagement with **compliance, records, privacy, security, and legal** stakeholders (Global Requirement 5) and an explicit, recorded **determination boundary** — the CoE governs method, naming, ownership, lifecycle, and the simulation-first gate; the stakeholders own retention schedules, records file plans, legal holds, insider-risk/communication-compliance case actions, and regulatory applicability. **Why:** The CoE must not silently make regulatory/records/legal determinations; the boundary keeps the governance operating model separate from the decisions it supports. **Verify:** Named approvers per capability are recorded in `Purview CoE Charter – Board`; the boundary is written into the charter. **Implement:** Identify owners and record the boundary before the CoE takes an artifact to *enforce*.

2.10 A pilot artifact in simulation/test

Description: A single **pilot artifact** (`Purview CoE Pilot – Stage1`) — for example one DLP or auto-labeling policy — taken through the full intake workflow in **simulation/test mode** to prove the gate before governing the estate (Global Requirement 6). **Why:** The intake gate must itself be validated end-to-end (Request → simulate → validate → board approval) before it is trusted for every future change. **Verify:** The pilot artifact is in simulation/test with recorded evidence; `OPSCOE-Validate.ps1` passes it against the standards. **Implement:** Choose a low-risk artifact, run it through intake in simulation, and only then treat the gate as production-ready.

Checklist

- Compliance licensing inventoried; in-remit vs out-of-scope capabilities recorded

- Compliance Administrator access confirmed; Global Admin reserved for one-time consent
- ExchangeOnlineManagement + Microsoft Graph SDK installed; `Connect-IPPSSession` / `Connect-MgGraph` succeed
- Graph read scopes consented (no write scopes needed for governance reads)
- CoE board named, separated, and mapped to built-in role groups; matrix CSV ready
- CoE standards agreed and captured as `Purview CoE Standards – Baseline`
- Intake workflow documented; change control and request template ready
- Governed-artifact inventory readable; portal-only capabilities recorded by attestation
- Stakeholders named per capability; determination boundary written into the charter
- `Purview CoE Pilot – Stage1` in simulation/test with recorded evidence

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Microsoft Purview Center of Excellence

Follow these steps **in order**. The model is **Assess** → **Design** → **Configure** → **Pilot (simulation)** → **Validate** → **Expand**. Governance role and tag changes are previewed with `-WhatIf` **before any** `-Commit`; no governed policy is taken to *enforce* without passing simulation/test and board approval. Each step names its artifacts, its pass/fail gate, and its rollback. Portal paths and cmdlets are cited as of authoring — **verify them in your own tenant**; where a capability is unlicensed or portal-only, govern it by attestation rather than inventing a cmdlet.

> ***Determination boundary (restated).*** *This guide configures the *operating model* — board, standards, intake, lifecycle. It does not author policy contents or make retention/records/legal or insider-risk determinations; those are deferred to compliance, records, privacy, and legal.*

Phase 0 — Assess (read-only)

Step 0.1 — Connect and confirm scope

Open Security & Compliance PowerShell and (for audit/role-membership cmdlets) Exchange Online:

```
Connect-IPPSSession
Connect-ExchangeOnline          # for Get-AdminAuditLogConfig / Add-RoleGroupMember
Connect-MgGraph -Scopes "Organization.Read.All","Directory.Read.All","User.Read.All","RoleManagement.Read.
```

Run the scope/health helper:

```
.\OPSCOE-ScopeCheck.ps1 -UseGraph
```

It confirms sign-in, reports the compliance SKUs present, lists which governed capabilities are visible (labels, DLP, retention, auto-labeling, communication compliance, audit), and flags unlicensed or portal-only capabilities. Output: `Purview CoE Scope – Assess` (CSV). **Gate:** sign-in OK and the in-remit capability list agreed. **FAIL** on any capability you intend to govern that the script cannot see → resolve licensing/roles before proceeding.

Step 0.2 — Inventory the estate the CoE will govern

```
.\OPSCOE-Discovery.ps1 -OutPath ".\Purview-CoE-Register-Governed.csv"
```

This enumerates every readable governed artifact — sensitivity labels, label policies, DLP policies + rules, retention policies/labels + rules, auto-labeling policies, and (where readable) communication-compliance and audit-retention policies — and for each records its **name**, **type**, **current state** (e.g. enabled/disabled/test/simulation where the object exposes it), and whether it **conforms to the naming standard**. Output: `Purview CoE Register – Governed` (the raw inventory that becomes the register). **Gate:** the inventory runs clean and every in-remit capability is represented (licensed+readable enumerated; portal-only flagged for attestation). Record the count and the non-conforming names — this is the baseline the CoE will bring into standard. **Rollback:** none — read-only.

Phase 1 — Design (author the operating model)

Step 1.1 — Author the CoE charter and board RACI

Write `Purview CoE Charter – Board`: the CoE's purpose, the **board membership** (chair + one representative per capability), the **RACI**, the **separation-of-duties** rules, the **determination boundary** (what the board governs vs what compliance/records/privacy/legal decide), and the **named approvers** per capability. Map each seat to a **built-in Purview role group** (Overview §"The CoE governance board"). **Gate:** every seat has a named owner and a mapped role group; insider-risk, communication-compliance, and eDiscovery seats are **separate** from the chair/compliance-administration seat.

Step 1.2 — Capture the standards baseline

Write `Purview CoE Standards – Baseline`: the **naming convention** (`<Purpose> – <Capability> – <State>`), the **ownership + review-cadence** rule (default quarterly), the **simulation/test-first** rule, the **change-control/versioning** rule, and the **separation-of-duties** rule. This file is the machine-checkable contract `OPSCOE-Validate.ps1` evaluates against (it reads the naming pattern and cadence from a small standards config). **Gate:** the baseline exists and states the naming pattern, cadence, and each gate unambiguously.

Step 1.3 — Define the intake workflow and the role-group matrix

Document the **intake workflow** (Request → Triage → Design → Pilot in simulation → Validate → Board approval → Enforce → Register), the `Purview CoE Intake – Request` template, and build the **role-group matrix CSV** (`Seat,RoleGroup,Member` rows) that `OPSCOE-ApplyStandards.ps1` consumes. **Gate:** the workflow has an owner per stage and the matrix names every board member and their role group.

Phase 2 — Configure (pilot-first, `-WhatIf` before `-Commit`)

Step 2.1 — Preview the board role-group membership

```
.\OPSCOE-ApplyStandards.ps1 -RoleMatrixCsv ".\coe-board-matrix.csv" -WhatIf
```

The script reads the matrix and shows exactly which members it would add to which built-in role groups (and which custom scoped groups it would create), printing the **reverse command** for each change. No change is made. **Gate:** the preview matches the charter exactly — least-privilege, separated, named. If it would add anyone to an investigative role group (insider risk / communication compliance / eDiscovery) who is not named in the charter, **STOP** and correct the matrix.

Step 2.2 — Apply the board role-group membership

```
.\OPSCOE-ApplyStandards.ps1 -RoleMatrixCsv ".\coe-board-matrix.csv" -Commit
```

Adds the named members to the built-in role groups (`Add-RoleGroupMember`) and creates any custom scoped groups (`New-RoleGroup`) defined in the matrix. Each applied change is logged with its reverse command. **Gate:** `Get-RoleGroupMember` for each seat resolves to exactly the named members. **Rollback:** run the printed reverse commands (`Remove-RoleGroupMember` / `Remove-RoleGroup`) to undo any membership or group creation.

Step 2.3 — (Optional) Stamp ownership/standard tags on governed artifacts

Where a governed object exposes a writable `Comment`/notes field (DLP, retention, auto-labeling policies via `Set-DlpCompliancePolicy` / `Set-RetentionCompliancePolicy` / `Set-AutoSensitivityLabelPolicy -Comment`), stamp a standardized ownership/standard tag so the owner and review date travel with the object:

```
.\OPSCOE-ApplyStandards.ps1 -StampOwnership -OwnerMapCsv ".\coe-owner-map.csv" -WhatIf   # preview
.\OPSCOE-ApplyStandards.ps1 -StampOwnership -OwnerMapCsv ".\coe-owner-map.csv" -Commit   # apply
```

The script only writes the `Comment` field (a non-disruptive metadata stamp of the form `CoE-Owner=<name>; Review=<date>`); it **never** changes a policy's rules, scope, or enforcement state. It guards `Get-Command` for each `Set-*` cmdlet and skips capabilities that are unlicensed or lack a writable comment surface. **Gate:** each stamped artifact's `Comment` shows the owner/review tag; no rule/scope/enforcement value changed (re-run `OPSCOE-Discovery.ps1` to confirm states are unchanged). **Rollback:** the script records each prior `Comment` value; re-run with `-Restore` to revert the stamps.

> ***The CoE never flips a policy to enforce.*** *Enforcement remains a board-gated, intake-driven action performed in the portal after simulation passes — not something these scripts do.*

Phase 3 — Pilot the intake gate (simulation/test)

Step 3.1 — Take one artifact through intake in simulation

Pick a low-risk **pilot artifact** (`Purview CoE Pilot – Stage1`) — e.g. one DLP or auto-labeling policy — and run it through the full intake workflow **in simulation/test mode**:

- **DLP:** create/configure the policy with **Test mode (with/without notifications)** — Purview portal

→ `Data loss prevention` → `Policies` → `New policy` → *"Test it out first"* (verify path/label in your tenant). Do **not** select *Turn it on right away*.

- **Auto-labeling:** create the policy in **Simulation** — `Information protection → Auto-labeling →

New` → run in simulation before turning on.

- **Retention:** stage as **disabled**/adaptive-scope-limited before broad application.

Name it to the standard (`<Purpose> – <Capability> – Simulation`), record the owner, and capture the simulation results as evidence. **Gate:** the pilot artifact exists **in simulation/test**, is standard-named, has an owner, and has recorded simulation evidence — and it is **not** enforced. **Rollback:** delete the simulation policy (it changed nothing in enforce; simulation does not act on content).

Step 3.2 — Board approval on the pilot

Run the pilot through **Triage** → **board approval** using the intake template; record the approval on `Purview CoE Intake – Request`. **Gate:** the board approval is recorded before the pilot could ever be enforced.

Phase 4 — Validate (see the Testing Guide for full procedures)

Step 4.1 — Validate every governance gate

```
.\OPSCOE-Validate.ps1 -StandardsConfig ".\coe-standards.json" -Register ".\Purview-CoE-Register-Governed.c
```

Checks: **naming conformance** of every artifact; **every artifact has an owner** (register or stamped `Comment`) and a **review date**; **no high-impact policy is enforced without recorded simulation evidence**; **separation of duties** intact (no board member in a conflicting investigative role); and the **register reconciles** to the live tenant (no orphaned or unregistered artifacts). Output: `Purview CoE Lifecycle – Review` with a per-artifact PASS/FAIL/WARN. **Gate:** all gates PASS (WARN allowed only with a recorded, board-accepted exception). Any FAIL is remediated via intake before the estate is declared governed. **Rollback:** none — read-only.

Step 4.2 — Produce the governance scorecard

```
.\OPSCOE-Register.ps1 -Register ".\Purview-CoE-Register-Governed.csv" -Validation ".\Purview-CoE-Lifecycle
```

Produces `Purview CoE Scorecard – Report`: counts by capability, % naming-conformant, % owned, % with review date, count enforced-without-evidence (target: 0), and the next-review-due list. **Gate:** the scorecard is the board's standing artifact and the audit answer.

Phase 5 — Expand (run the CoE as a standing rhythm)

Step 5.1 — Bring the full estate under the register and cadence

Assign an owner and a review date to **every** in-remit artifact (remediating non-conforming names via intake), register portal-only capabilities (insider risk) by attestation, and set the review cadence (default quarterly). **Gate:** 100% of in-remit artifacts owned and review-dated in the register.

Step 5.2 — Operate intake, lifecycle review, and drift detection

- **Intake:** every new label/DLP/retention/auto-label/insider-risk request enters via the workflow;

nothing reaches enforce without simulation evidence and board approval.

- **Lifecycle review:** on the cadence, re-run `OPSCOE-Discovery.ps1` + `OPSCOE-Validate.ps1` and review

the `Purview CoE Lifecycle – Review` output at the board; decommission orphaned/unowned artifacts under change control (identify → confirm owner → disable → remove).

• **Drift detection:** compare each cadence's discovery against the registered baseline; new, changed, or orphaned artifacts are flagged and routed to intake. **Gate:** each review cycle closes with an updated register, an updated scorecard, and zero enforced-without-evidence artifacts.

Rollback summary

Change	Reverse
Role-group membership added (2.2)	`OPSCOE-ApplyStandards.ps1` printed `Remove-RoleGroupMember` / `Remove-RoleGroup` reverse commands
Ownership/standard `Comment` stamp (2.3)	`OPSCOE-ApplyStandards.ps1 -Restore` (reverts to recorded prior `Comment`)
Pilot simulation policy (3.1)	Delete the simulation/test policy (no content was acted on)
Register/scorecard files (0.2, 4.x)	Delete the CSVs — read-only artifacts

No step in this guide enforces a Purview policy, changes a policy's rules or scope, or makes a regulatory/records/legal determination. Enforcement is always a separate, board-gated, intake-driven portal action taken after simulation passes.

5. Testing & Validation Guide — Microsoft Purview Center of Excellence

Every governance gate has a **pass/fail** test. Run these after Phase 2 (Configure) and again on every lifecycle review cycle. Read-only scripts are safe to re-run at any time; nothing here enforces a policy. Record results on `Purview CoE Lifecycle – Review` and roll the summary into `Purview CoE Scorecard – Report`.

> ***What "pass" means for a governance CoE.*** *The CoE is validated when the *operating model* holds: the board is least-privilege and separated, every governed artifact is named to standard, owned, and review-dated, no high-impact policy is enforced without simulation evidence, and the register reconciles to the live tenant. The CoE does **not** validate the correctness of a policy's regulatory content — that is the stakeholders' determination.*

Test 1 — Scope and toolchain (Requirement 2.1–2.4)

Procedure: `.\OPSCOE-ScopeCheck.ps1 -UseGraph` **Pass:** sign-in OK; the in-remit capability list matches the agreed remit; unlicensed/portal-only capabilities are correctly flagged. **Fail:** any capability you intend to govern is not visible → resolve licensing/roles (2.1–2.2) and re-run.

Test 2 — Estate inventory completeness (Requirement 2.8)

Procedure: `.\OPSCOE-Discovery.ps1` **Pass:** the inventory enumerates every licensed+readable governed capability; portal-only capabilities are flagged for attestation; the artifact count and non-conforming-name list are recorded. **Fail:** an in-remit, licensed capability returns no data because of a missing read role → assign the least-privilege read role and re-run.

Test 3 — Board least-privilege and separation of duties (Requirement 2.5)

Procedure: `Get-RoleGroupMember` for each seat (or re-run `.\OPSCOE-ApplyStandards.ps1 -WhatIf` to diff intent vs live), and `.\OPSCOE-Validate.ps1` (separation check). **Pass:** each role group resolves to exactly the named members from the charter; **no** board member holds both governance administration and an investigative role (insider risk / communication compliance / eDiscovery); investigative role membership is minimal and named. **Fail:** an unexpected member, or a chair/compliance-admin account also in an investigative role → correct membership via the matrix and re-run `-WhatIf` then `-Commit`.

Test 4 — Naming-standard conformance (Requirement 2.6)

Procedure: `.\OPSCOE-Validate.ps1 -StandardsConfig .\coe-standards.json -Register .\Purview-CoE-Register-Governed.csv` **Pass:** every governed artifact name matches the standard pattern `<Purpose> – <Capability> – <State>` (or is on a recorded, board-accepted remediation list). **Fail:** any non-conforming name not on the remediation list → route through intake to rename (recreate under the standard; retiring the old artifact via change control) and re-run.

Test 5 — Ownership and review-date coverage (Requirement 2.6)

Procedure: `.\OPSCOE-Validate.ps1` (ownership + review-date checks) **Pass:** 100% of in-remit artifacts have a named owner (register or stamped `Comment`) and a review date within the cadence window. **Fail:** any orphaned (no owner) or overdue artifact → assign an owner / schedule review, or route to decommission if genuinely orphaned.

Test 6 — Simulation-first gate (Requirement 2.10, the highest-value test)

Procedure: `.\OPSCOE-Validate.ps1` (enforced-without-evidence check) — cross-references each **enforced** high-impact policy (DLP, auto-labeling, retention, insider risk) against a recorded simulation-evidence entry. **Pass: zero** enforced high-impact policies lack recorded simulation/test evidence and board approval. **Fail:** any enforced policy with no simulation evidence → treat as a governance incident: record it, have the owner produce/attach evidence retroactively, and add the gap to the board review. (You do not disable a live control blindly — that itself is a change; route the remediation through intake.)

Test 7 — Intake gate end-to-end on the pilot (Requirement 2.10)

Procedure: Verify `Purview CoE Pilot – Stage1` went Request → simulation → validate → board approval, with the artifact **in simulation/test** (not enforced) and evidence recorded. **Pass:** the pilot demonstrates the full gate; the board approval predates any possible enforcement. **Fail:** any stage skipped (no simulation, no approval record) → the gate is not yet trustworthy; re-run the pilot correctly before governing the estate.

Test 8 — Register reconciles to the live tenant (drift)

Procedure: `.\OPSCOE-Validate.ps1` (reconciliation) — diffs the register against a fresh `OPSCOE-Discovery.ps1` run. **Pass:** no unregistered artifacts in the tenant and no registered artifacts missing from the tenant; any delta is explained (new intake, approved decommission). **Fail:** an artifact exists in the tenant but not the register (drift / out-of-band change) → route to intake and register it; an artifact in the register is gone from the tenant → confirm approved decommission or investigate.

Test 9 — Scorecard produced and reconciles (Requirement 2.9)

Procedure: `.\OPSCOE-Register.ps1 -Register ... -Validation ...` **Pass:** `Purview CoE Scorecard – Report` produces counts by capability, % naming-conformant, % owned, % review-dated, enforced-without-evidence = 0, and a next-review-due list — and the totals reconcile to the discovery count. **Fail:** totals do not reconcile → re-run discovery/validation and rebuild before the board review.

Regression / cadence checklist (run every review cycle)

- Test 1 scope/toolchain — pass
- Test 2 inventory complete — pass
- Test 3 board least-privilege + separation — pass
- Test 4 naming conformance — pass (or remediation list current)
- Test 5 ownership + review-date coverage 100%
- Test 6 simulation-first gate — **enforced-without-evidence = 0**
- Test 7 intake gate proven (pilot / latest change)
- Test 8 register reconciles (no unexplained drift)
- Test 9 scorecard produced and reconciles
- `Purview CoE Lifecycle – Review` + `Purview CoE Scorecard – Report` filed with the board

6. Troubleshooting — Microsoft Purview Center of Excellence

The failure modes that make a Purview **CoE stall, over-reach, or fail its own audit** — plus the role-group, ownership-stamp, validation, and reconciliation issues you'll hit standing up and running the operating model. Each: **symptom** → **likely cause** → **fix.** The CoE governs the *operating model* (board, standards, intake, lifecycle); it **never flips a policy to enforce** and makes **no** retention/records/ legal or insider-risk determination — those are deferred. Every write tool previews with `-WhatIf` first.

`OPSCOE-ScopeCheck.ps1` can't see a capability you intend to govern

- **Cause:** the capability is unlicensed, or portal-only in your tenant (insider risk especially).
- **Fix:** confirm the SKU (Requirement 2.1). If genuinely portal-only, govern it by **attestation** — a

register row with a named owner and review date — rather than inventing a cmdlet. The gate FAILS only for a capability you intend to govern that you also can't see with a correct license/role.

`OPSCOE-ApplyStandards.ps1 -RoleMatrixCsv` prints `[MISSING] role group`

- **Cause:** the role-group name in the matrix doesn't exist — Purview role-group names **evolve** and many

tenants use **unified Purview RBAC** with different names.

- **Fix:** verify the current name in **Settings** → **Roles & scopes** → **Permissions**; fix the matrix. To create

a *custom scoped* group deliberately, mark the row `Custom=Yes` (the tool then `New-RoleGroups` it). Do not guess built-in names.

The preview would add someone to an investigative role

- **Cause:** the matrix grants Insider Risk / Communication Compliance / eDiscovery to a member not named in

the charter.

- **Fix:** the `[WARN] … INVESTIGATIVE role` line is the gate — **STOP** and confirm the member is named in

the CoE charter and that separation of duties holds (investigative seats separate from the chair/compliance administration seat). Correct the matrix before `-Commit`.

The ownership stamp changed (or seems to have changed) a policy

- **Cause:** concern that `-StampOwnership` altered a policy's rules/scope/enforcement.
- **Fix:** it does **not** — it writes only the `Comment` field (`CoE-Owner=<name> Review=<date>`),

preserving any non-CoE comment text. Re-run `OPSCOE-Discovery.ps1` and confirm every policy's **State** is unchanged. **Rollback:** `OPSCOE-ApplyStandards.ps1 -StampOwnership -Restore -Commit` reverts each Comment from the saved backup (`OPSCOE-StampBackup.json`).

Stamp skips a capability with `[SKIP] … cmdlets unavailable`

- **Cause:** that capability is unlicensed or lacks a writable comment surface (`Set-*` cmdlet absent).
- **Fix:** expected behavior — the tool guards `Get-Command` and skips rather than inventing a surface.

Govern that capability's ownership in the register by attestation instead.

`OPSCOE-Validate.ps1` reports FAIL on naming

- **Cause:** an artifact doesn't match `<Purpose> – <Capability> – <State>`.
- **Fix:** route the rename through **intake** (renaming is per-object in the portal). Portal-only/attestation

rows are exempt (reported WARN, not FAIL). Adjust the `namePattern` in `coe-standards.json` only if your agreed standard genuinely differs — don't loosen it to hide drift.

Validate reports FAIL on `EnforceEvidence`

- **Cause:** a high-impact policy (DLP / auto-label / retention) is in an **enforced** state with **no recorded simulation/test evidence** in the register.
- **Fix:** this is the core CoE gate. Either record the simulation evidence (`Evidence=/SimEvidence` in the register Detail) if the pilot genuinely passed, or return the policy to **simulation/test** and take it back through intake. The board's target is **enforced-without-evidence = 0**.

Validate reports FAIL on `Owner` / `ReviewDate`

- **Cause:** the artifact has no owner or no (valid, current) review date.
- **Fix:** assign an owner and a review date — in the register or via an `OPSCOE-ApplyStandards.ps1` ownership stamp. An overdue review is a **WARN** (past the cadence in `reviewCadenceDays`); a missing one is a **FAIL**. Bring the estate to 100% owned and review-dated.

Separation-of-duties FAIL on the board matrix

- **Cause:** one member holds **two** investigative roles (or an investigative role plus others without a recorded exception).
- **Fix:** remove one membership (`OPSCOE-ApplyStandards.ps1` printed the reverse `Remove-RoleGroupMember`) or record a board-accepted exception for a WARN. Insider risk, communication compliance, and eDiscovery stay separated from the chair/compliance-administration seat.

Reconciliation reports `Unregistered` / orphaned live artifacts

- **Cause:** a policy exists in the tenant that isn't in the register — an out-of-band change bypassing intake.
- **Fix:** route it through intake — assign an owner, standard name, and review date, and register it; or decommission it under change control (identify → confirm owner → disable → remove). This is exactly the drift the CoE exists to catch.

`OPSCOE-Register.ps1` scorecard shows enforced-without-evidence > 0

- **Cause:** the validation output still contains `EnforceEvidence` FAILs.
- **Fix:** remediate each in validation first (record evidence or de-enforce), re-run `OPSCOE-Validate.ps1`, then regenerate the scorecard. The scorecard is the board's audit answer — it must show **0** before sign-off.

`Connect-IPPSSession` / `Connect-ExchangeOnline` / `Connect-MgGraph` fails

- **Cause:** module not installed, or MFA/modern-auth not completed.
- **Fix:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser` (+ `Microsoft.Graph`); complete the

interactive sign-in (Requirement 2.3). The read-only scripts are safe to retry; write tools are `-WhatIf` by default.

When to escalate beyond this solution

- **What a policy should *contain* or *require*** — retention schedules, records file plans, insider-risk

conditions, regulatory applicability → **compliance, records, privacy, and legal**. The CoE governs the operating model; it does not make these determinations.

- **The decision to enforce** → the board, via intake, **after** simulation passes — never a script action.
- **A role group or portal surface that has genuinely changed** → verify against current Microsoft

documentation; Purview RBAC and the portal evolve, and unified RBAC may be in effect.

7. Best Practices — Microsoft Purview Center of Excellence

Field-proven recommendations for running a Purview CoE that stays useful instead of becoming shelfware.

Govern the operating model, not the determinations

- The CoE's job is **board, standards, intake, lifecycle, evidence** — not retention schedules, records

file plans, legal holds, or insider-risk case outcomes. Keep the **determination boundary** written into the charter and referenced whenever a request blurs it. The moment the CoE starts *deciding* regulatory questions instead of *governing how they are implemented*, it has overreached.

- Every governed artifact should trace to a **stakeholder-owned requirement** (a compliance obligation,

a records schedule, a data-protection intent) — the CoE ensures it is named, owned, simulated, and reviewed; the stakeholder owns *why it exists*.

Make simulation-first non-negotiable

- The single most valuable rule in the whole CoE is: **no DLP, auto-labeling, retention, or

insider-risk policy is enforced without recorded simulation/test evidence and board approval.** Most catastrophic Purview incidents (blocked mail, mass encryption, unexpected deletion, employee flags) come from skipping simulation.

- Keep the **evidence** with the artifact, not in someone's inbox — a register entry linking to

simulation results and the board approval is the audit answer. `enforced-without-evidence = 0` is the CoE's flagship metric.

Least privilege and separation of duties, enforced on the CoE itself

• The CoE preaches least privilege — so **run it that way**. The chair holds Compliance Administrator, not Global Administrator; investigative seats (insider risk, communication compliance, eDiscovery) are **separate**, minimal, and named. A CoE administered from a blanket Global Admin has no credibility.

• Prefer **built-in role groups**; create custom scoped groups only where you genuinely need tighter scope. Fewer, well-understood groups beat a sprawl of bespoke ones.

• For insider risk and communication compliance, apply **privacy-by-design** (pseudonymization where the capability supports it) in that capability's own solution, and defer *who may hold the role* to HR, privacy, and legal.

Standardize names so the estate reads itself

• Enforce `<Purpose> – <Capability> – <State>` on every artifact. A name like `Financial Data – DLP – Simulation` tells you what it protects and that it is not yet enforced — without opening it.

• Renaming a live policy is a **change**, not a cleanup — route it through intake, recreate under the standard, and retire the old one under change control. Never bulk-rename enforced policies out of band.

Stamp ownership onto the object where you can

• Use the non-disruptive `Comment` stamp (`CoE-Owner=…; Review=…`) on DLP/retention/auto-labeling policies so the owner and review date travel with the object and survive a register mishap. Never let the stamp script touch rules, scope, or enforcement state — it writes metadata only.

• For portal-only capabilities (insider risk especially), govern by **attestation** in the register with a named owner — do not invent a cmdlet to enumerate what the platform does not expose.

Run a real cadence, or the register rots

• A register that is never reconciled is worse than none — it lies. Run **discovery + validate on a fixed cadence **(default quarterly), review the output** at the board**, and act on drift.

• **Drift detection** is the early-warning system: any artifact in the tenant but not the register is an out-of-band change; any registered artifact gone from the tenant is an unrecorded decommission. Both route to intake.

• **Decommission deliberately**: identify orphaned/unowned → confirm no owner claims it → disable → observe → remove, all under change control. Deleting a retention or DLP policy can have compliance consequences — confirm with the stakeholder first.

Keep the CoE lightweight enough to survive

• The intake workflow must be **fast for low-risk changes** or people will route around it. Triage should fast-track obviously low-risk requests and reserve full board review for high-impact ones — a gate everyone bypasses governs nothing.

• Automate the boring parts (discovery, validation, scorecard) so board time is spent on **decisions**, not data gathering. The scripts exist so the board reviews a scorecard, not a raw tenant.

Assume the portal will change

• Microsoft is actively evolving the Purview portal and moving role management toward a unified Purview RBAC model. **Verify portal paths and cmdlet availability each cycle**; the scripts guard `Get-Command` and skip what is unavailable rather than failing. When a path moves, update the charter and standards references — do not assume last quarter's screenshot still holds.

Protect the evidence and the credentials

• Never store credentials or secrets in scripts or the register — every script uses interactive modern-auth. The register and scorecard may list policy **names and owners**; they must **not** contain policy contents that are themselves sensitive, nor any employee case data.

• Treat the governance artifacts (charter, register, scorecard, evidence) as the **audit trail** — version them, retain them per your own records schedule, and keep them where the board (not the whole tenant) can read them.

8. Deliverables Summary — Microsoft Purview Center of Excellence

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The CoE operating model — governance board (least-privilege, separated), standards, intake workflow, and lifecycle cadence — plus the naming convention, limitations, and determination boundary
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Purview)	Category-wide prerequisites, published once

Document	Purpose
3. Implementation Guide	Exact, ordered portal + S&C PowerShell + Graph steps: assess the estate, author charter/standards/intake, apply the board role model and standard tags pilot-first (`-WhatIf` → `-Commit`), pilot the intake gate in simulation, validate every gate, expand to the full estate and run the cadence
5. Testing Guide	Nine pass/fail governance-gate tests + a cadence checklist
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`OPSCOE-ScopeCheck.ps1`	Read-only scope/health helper: confirms S&C PowerShell / Exchange Online / Graph sign-in, reports compliance SKUs present, and lists which governed capabilities are visible vs unlicensed/portal-only. Writes `Purview CoE Scope – Assess`
`OPSCOE-Discovery.ps1`	Read-only estate inventory: enumerates every readable governed artifact (labels, label policies, DLP policies+rules, retention policies/labels+rules, auto-labeling, and where readable communication-compliance / audit-retention), with name, type, state, and naming-standard conformance. Writes `Purview CoE Register – Governed`
`OPSCOE-ApplyStandards.ps1`	Configuration/remediation helper: populates the CoE board's built-in role groups from a matrix (creating custom scoped groups where needed) and optionally stamps a non-disruptive `CoE-Owner/Review` tag onto policy `Comment` fields. `-WhatIf` **by default**, `-Commit` to apply, `-Restore` to revert stamps; prints the reverse command for every change. Never changes policy rules/scope/enforcement
`OPSCOE-Validate.ps1`	Read-only governance-gate validation: naming conformance, ownership + review-date coverage, the simulation-first (enforced-without-evidence) gate, separation of duties, and register-to-tenant reconciliation. Writes `Purview CoE Lifecycle – Review` with per-artifact PASS/FAIL/WARN

Script	What it does
`OPSCOE-Register.ps1`	Read-only reporting: builds the governance scorecard — counts by capability, % naming-conformant, % owned, % review-dated, enforced-without-evidence count, and next-review-due list. Writes `Purview CoE Scorecard – Report`

All scripts are **safe by default** — read-only, or a `-WhatIf` preview requiring an explicit `-Commit` to change anything (and `ApplyStandards` writes only role-group membership and a metadata `Comment` stamp — never a policy's rules, scope, or enforcement state). No credentials or secrets are stored in code; every script uses interactive modern-auth (`Connect-IPPSSession` / `Connect-ExchangeOnline` / `Connect-MgGraph`) and guards `Get-Command` so unlicensed or portal-only capabilities are skipped, not failed.

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable governance operating model following the model: **Assess → Design → Configure → Pilot →

Validate → Expand.**

Outcome

A standing **Purview Center of Excellence** — a named, least-privilege, separated governance board; written standards (naming, ownership, simulation-first, change-control, separation of duties); a single intake gate that keeps ungoverned and un-simulated policies out of the tenant; and a lifecycle cadence with drift detection and a decommission path — so Purview governance is institutionalized, every governed artifact is owned and defensible, and the audit answer is one scorecard export. The CoE governs the method and the evidence; regulatory, records, and legal determinations remain with the organization's compliance, records, privacy, and legal functions.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OPSCOE-ApplyStandards.ps1

Purpose: Two governance actions for a Purview Center of Excellence, both SAFE BY DEFAULT (preview with -WhatIf, apply only with -Commit): (1) -RoleMatrixCsv: reads a Seat,RoleGroup,Member matrix and adds the named members to the built-in role groups (Add-RoleGroupMember), creating any custom scoped groups (New-RoleGroup) the matrix declares. Every applied change prints its REVERSE command. (2) -StampOwnership -OwnerMapCsv: writes a non-disruptive ownership/standard tag into the writable Comment field of governed policies (CoE-Owner=<name>; Review=<date>) via Set-DlpCompliancePolicy / Set-RetentionCompliancePolicy / Set-AutoSensitivityLabelPolicy -Comment. It NEVER changes a policy's rules, scope, or enforcement state, and records each prior Comment so -Restore reverts it. This tool never flips a policy to enforce - enforcement is a board-gated, intake-driven portal action. HOW TO RUN Board role-group membership: .\OPSCOE-ApplyStandards.ps1 -RoleMatrixCsv .\coe-board-matrix.csv -WhatIf # preview .\OPSCOE-ApplyStandards.ps1 -RoleMatrixCsv .\coe-board-matrix.csv -Commit # apply Ownership/standard stamp: .\OPSCOE-ApplyStandards.ps1 -StampOwnership -OwnerMapCsv .\coe-owner-map.csv -WhatIf .\OPSCOE-ApplyStandards.ps1 -StampOwnership -OwnerMapCsv .\coe-owner-map.csv -Commit Revert the stamps: .\OPSCOE-ApplyStandards.ps1 -StampOwnership -Restore -Commit matrix columns: Seat,RoleGroup,Member (+ optional Custom=Yes to create a scoped group). owner-map columns: Type,Name,Owner,Review (Type = Dlp|Retention|AutoLabel). Prereqs: ExchangeOnlineManagement (Connect-IPPSSession) + Connect-ExchangeOnline for role-group cmdlets. EXPECTED RESULTS Without -Commit: a per-change plan and, for role adds, the reverse command. With -Commit: each change applied with a PASS/FAIL line. PASS = matrix members resolve to exactly the intended role groups; each stamped policy's Comment shows the owner/review tag and NO rule/scope/enforcement value changed. .NOTES White-label Microsoft Purview tooling. -WhatIf by default; writes only with -Commit. Security & Compliance PowerShell / Exchange Online. No credentials in code. Role-group names EVOLVE - verify in your tenant. > [CmdletBinding()] param([string]$RoleMatrixCsv, [switch]$StampOwnership, [string]$OwnerMapCsv, [switch]$Restore, [string]$RestoreStorePath = ".\OPSCOE-StampBackup.json", [switch]$Commit)

How to run: Board role-group membership: .\OPSCOE-ApplyStandards.ps1 -RoleMatrixCsv .\coe-board-matrix.csv -WhatIf # preview .\OPSCOE-ApplyStandards.ps1 -RoleMatrixCsv .\coe-board-matrix.csv -Commit # apply Ownership/standard stamp: .\OPSCOE-ApplyStandards.ps1 -StampOwnership -OwnerMapCsv .\coe-owner-map.csv -WhatIf .\OPSCOE-ApplyStandards.ps1 -StampOwnership -OwnerMapCsv .\coe-owner-map.csv -Commit Revert the stamps: .\OPSCOE-ApplyStandards.ps1 -StampOwnership -Restore -Commit matrix columns: Seat,RoleGroup,Member (+ optional Custom=Yes to create a scoped group). owner-map columns: Type,Name,Owner,Review (Type = Dlp|Retention|AutoLabel). Prereqs: ExchangeOnlineManagement (Connect-IPPSSession) + Connect-ExchangeOnline for role-group cmdlets. EXPECTED RESULTS Without -Commit: a per-change plan and, for role adds, the reverse command. With -Commit: each change applied with a PASS/FAIL line. PASS = matrix members resolve to exactly the intended role groups; each stamped policy's Comment shows the owner/review tag and NO rule/scope/enforcement value changed. .NOTES White-label Microsoft Purview tooling. -WhatIf by default; writes only with -Commit. Security & Compliance PowerShell / Exchange Online. No credentials in code. Role-group names EVOLVE - verify in your tenant. > [CmdletBinding()] param([string]$RoleMatrixCsv, [switch]$StampOwnership,

[string]$OwnerMapCsv, [switch]$Restore, [string]$RestoreStorePath = ".\OPSCOE-StampBackup.json", [switch]$Commit)

OPSCOE-Discovery.ps1

Purpose: Read-only estate inventory for a Purview Center of Excellence. Enumerates every readable governed artifact — sensitivity labels, label policies, DLP policies (+ rule counts), retention policies/labels (+ rule counts), auto-labeling policies, and (where readable) communication-compliance policies — and for each records its name, type, current state, owner (parsed from a CoE-Owner Comment stamp if present), and whether it conforms to the naming standard <Purpose> - <Capability> - <State>. This inventory becomes the governance register. HOW TO RUN PowerShell 5.1+ or 7+ with ExchangeOnlineManagement (Security & Compliance PowerShell): Connect-IPPSSession .\OPSCOE-Discovery.ps1 -OutPath ".\Purview-CoE-Register-Governed.csv" Optional: -NamePattern to override the default naming-standard regex. EXPECTED RESULTS One row per governed artifact with Type/Name/State/Owner/Conforms, plus a summary count and a non-conforming list. Written to the -OutPath CSV (default .\Purview-CoE-Register-Governed.csv). Resolves Requirement 2.8. This is the register baseline for OPSCOE-Validate.ps1 / -Register.ps1. .NOTES White-label Microsoft Purview tooling. Read-only — enumerates only. No credentials stored in code. Availability caveat: unlicensed or portal-only capabilities (insider risk especially) will not enumerate — they are governed by attestation in the register, not invented here. Cmdlets are standard Security & Compliance PowerShell; none are invented. Portal/cmdlet surfaces evolve — verify in your tenant. > [CmdletBinding()] param([string]$OutPath = ".\Purview-CoE-Register-Governed.csv", Default naming standard: "<Purpose> - <Capability> - <State>" (en dash or hyphen, 2 separators). [string]$NamePattern = '^.+\s[–-]\s.+\s[–-]\s.+$')

How to run: PowerShell 5.1+ or 7+ with ExchangeOnlineManagement (Security & Compliance PowerShell): Connect-IPPSSession .\OPSCOE-Discovery.ps1 -OutPath ".\Purview-CoE-Register-Governed.csv" Optional: -NamePattern to override the default naming-standard regex. EXPECTED RESULTS One row per governed artifact with Type/Name/State/Owner/Conforms, plus a summary count and a non-conforming list. Written to the -OutPath CSV (default .\Purview-CoE-Register-Governed.csv). Resolves Requirement 2.8. This is the register baseline for OPSCOE-Validate.ps1 / -Register.ps1. .NOTES White-label Microsoft Purview tooling. Read-only — enumerates only. No credentials stored in code. Availability caveat: unlicensed or portal-only capabilities (insider risk especially) will not enumerate — they are governed by attestation in the register, not invented here. Cmdlets are standard Security & Compliance PowerShell; none are invented. Portal/cmdlet surfaces evolve — verify in your tenant. > [CmdletBinding()] param([string]$OutPath = ".\Purview-CoE-Register-Governed.csv", Default naming standard: "<Purpose> - <Capability> - <State>" (en dash or hyphen, 2 separators). [string]$NamePattern = '^.+\s[–-]\s.+\s[–-]\s.+$')

OPSCOE-Register.ps1

Purpose: Rolls the governance register (OPSCOE-Discovery.ps1) and the lifecycle-review results (OPSCOE-Validate.ps1) into the board's standing artifact: "Purview CoE Scorecard - Report" - counts by capability, % naming-conformant, % owned, % with a review date, count enforced-without-evidence (target 0), and the next-review-due list. Read-only - aggregates the two inputs, changes nothing. HOW TO RUN .\OPSCOE-Register.ps1 -Register .\Purview-CoE-Register-Governed.csv -Validation .\Purview-CoE-Lifecycle-Review.csv -OutPath .\Purview-CoE-Scorecard-Report.csv Optional: -ReviewCadenceDays 90 to set the "next review due" horizon. EXPECTED RESULTS A per-capability scorecard CSV plus a printed summary. The board's audit answer: enforced-without- evidence should be 0; naming/owned/review percentages trend to 100%. PASS = scorecard produced and

enforced-without-evidence = 0. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials in code. Portal-only capabilities are counted as attestation rows. Verify capability availability/cmdlets in your tenant. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Register, [Parameter(Mandatory)][string]$Validation, [string]$OutPath = ".\Purview-CoE-Scorecard-Report.csv", [int]$ReviewCadenceDays = 90)

How to run: .\OPSCOE-Register.ps1 -Register .\Purview-CoE-Register-Governed.csv -Validation .\Purview-CoE-Lifecycle-Review.csv -OutPath .\Purview-CoE-Scorecard-Report.csv Optional: -ReviewCadenceDays 90 to set the "next review due" horizon. EXPECTED RESULTS A per-capability scorecard CSV plus a printed summary. The board's audit answer: enforced-without- evidence should be 0; naming/owned/review percentages trend to 100%. PASS = scorecard produced and enforced-without-evidence = 0. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials in code. Portal-only capabilities are counted as attestation rows. Verify capability availability/cmdlets in your tenant. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Register, [Parameter(Mandatory)][string]$Validation, [string]$OutPath = ".\Purview-CoE-Scorecard-Report.csv", [int]$ReviewCadenceDays = 90)

OPSCOE-ScopeCheck.ps1

Purpose: Read-only pre-flight for a Microsoft Purview Center of Excellence. Confirms your Security & Compliance PowerShell (and optional Graph) sign-in, reports which compliance SKUs are present, and lists which governed capabilities (sensitivity/retention labels, DLP, auto-labeling, communication compliance, audit) are visible versus unlicensed or portal-only — so the CoE's remit is bounded by what the tenant can actually run. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+, with the ExchangeOnlineManagement (Security & Compliance PowerShell) and, optionally, Microsoft.Graph modules: Connect-IPPSSession Connect-ExchangeOnline # optional, enables audit-config visibility .\OPSCOE-ScopeCheck.ps1 To also read licensing/directory via Graph, pass -UseGraph (Connect-MgGraph is invoked read-only). EXPECTED RESULTS A readiness summary: sign-in OK, compliance SKUs present, and per-capability visibility (in-remit / unlicensed / portal-only). Written to .\OPSCOE-ScopeCheck-Scope-Assess.csv. Resolves Requirements 2.1-2.4. Any FAIL must be resolved before governing that capability. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials are stored or requested in code. Availability caveat: Purview capabilities are licensing-/tenant-dependent and the portal evolves; insider risk (and parts of communication compliance) are primarily portal-managed with limited PowerShell — this script reports what it can see and flags the rest for attestation. Cmdlets are standard Security & Compliance PowerShell / Exchange Online / Graph; none are invented. > [CmdletBinding()] param([switch]$UseGraph, [string]$OutPath = ".\OPSCOE-ScopeCheck-Scope-Assess.csv")

How to run: PowerShell 5.1+ or 7+, with the ExchangeOnlineManagement (Security & Compliance PowerShell) and, optionally, Microsoft.Graph modules: Connect-IPPSSession Connect-ExchangeOnline # optional, enables audit-config visibility .\OPSCOE-ScopeCheck.ps1 To also read licensing/directory via Graph, pass -UseGraph (Connect-MgGraph is invoked read-only). EXPECTED RESULTS A readiness summary: sign-in OK, compliance SKUs present, and per-capability visibility (in-remit / unlicensed / portal-only). Written to .\OPSCOE-ScopeCheck-Scope-Assess.csv. Resolves Requirements 2.1-2.4. Any FAIL must be resolved before governing that capability. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials are stored or requested in code. Availability caveat: Purview capabilities are licensing-/tenant-dependent and the portal evolves; insider risk (and parts of communication compliance) are primarily portal-managed with limited PowerShell — this script reports what it can see and flags the rest for attestation. Cmdlets are standard Security & Compliance PowerShell / Exchange Online / Graph;

none are invented. > [CmdletBinding()] param([switch]$UseGraph, [string]$OutPath = ".\OPSCOE-ScopeCheck-Scope-Assess.csv")

OPSCOE-Validate.ps1

Purpose: The machine-checkable governance gate for a Purview Center of Excellence. Reads the standards config (naming pattern + review cadence) and the governance register (from OPSCOE-Discovery.ps1) and checks, per artifact: (1) NAMING conformance to <Purpose> - <Capability> - <State>; (2) an OWNER and a REVIEW date exist (register column or stamped Comment); (3) no HIGH-IMPACT policy is ENFORCED without recorded simulation/test evidence; (4) SEPARATION OF DUTIES - no board member sits in a conflicting investigative role; and (5) the register RECONCILES to the live tenant (no orphaned/unregistered artifacts). Read-only. Produces "Purview CoE Lifecycle - Review" with a per-artifact PASS/FAIL/WARN. HOW TO RUN Connect-IPPSSession # (+ Connect-ExchangeOnline for role-group membership reconciliation) .\OPSCOE-Validate.ps1 -StandardsConfig .\coe-standards.json -Register .\Purview-CoE-Register-Governed.csv -OutPath .\Purview-CoE-Lifecycle-Review.csv Optional: -BoardMatrixCsv .\coe-board-matrix.csv to run the separation-of-duties check; -SkipLiveReconcile to validate the register alone without connecting. coe-standards.json: { "namePattern": "...", "reviewCadenceDays": 90, "investigativeRoles": [...] } EXPECTED RESULTS A per-artifact PASS/FAIL/WARN table written to -OutPath, plus a summary. PASS overall = every gate PASS (WARN allowed only with a recorded, board-accepted exception). Any FAIL is remediated via intake before the estate is declared governed. .NOTES White-label Microsoft Purview tooling. Read-only - evaluates only. No credentials in code. Portal-only capabilities (insider risk) are governed by attestation rows, not invented cmdlets. Verify in your tenant. > [CmdletBinding()] param([Parameter(Mandatory)][string]$StandardsConfig, [Parameter(Mandatory)][string]$Register, [string]$OutPath = ".\Purview-CoE-Lifecycle-Review.csv", [string]$BoardMatrixCsv, [switch]$SkipLiveReconcile)

How to run: Connect-IPPSSession # (+ Connect-ExchangeOnline for role-group membership reconciliation) .\OPSCOE-Validate.ps1 -StandardsConfig .\coe-standards.json -Register .\Purview-CoE-Register-Governed.csv -OutPath .\Purview-CoE-Lifecycle-Review.csv Optional: -BoardMatrixCsv .\coe-board-matrix.csv to run the separation-of-duties check; -SkipLiveReconcile to validate the register alone without connecting. coe-standards.json: { "namePattern": "...", "reviewCadenceDays": 90, "investigativeRoles": [...] } EXPECTED RESULTS A per-artifact PASS/FAIL/WARN table written to -OutPath, plus a summary. PASS overall = every gate PASS (WARN allowed only with a recorded, board-accepted exception). Any FAIL is remediated via intake before the estate is declared governed. .NOTES White-label Microsoft Purview tooling. Read-only - evaluates only. No credentials in code. Portal-only capabilities (insider risk) are governed by attestation rows, not invented cmdlets. Verify in your tenant. > [CmdletBinding()] param([Parameter(Mandatory)][string]$StandardsConfig, [Parameter(Mandatory)][string]$Register, [string]$OutPath = ".\Purview-CoE-Lifecycle-Review.csv", [string]$BoardMatrixCsv, [switch]$SkipLiveReconcile)